Lab 5 – Static Code Analysis

NAME: Raksha V SRN: PES2UG24CS396 SEC: G

Pylint output:

Your code has been rated at 10.00/10 (previous run: 9.87/10, +0.13)

Bandit output:

Run started:2026-10-16 04:25:10.692891+00:00

Test results:

>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.

  Severity: Low Confidence: High

  CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

  More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess

  Location: .\blood_donation.py:6:0

5 import shutil

6 import subprocess

7 import tempfile

>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.

  Severity: Low Confidence: High

  CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

  More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subprocess_without_shell_equals_true.html

  Location: .\blood_donation.py:120:4

119 check=True,

```
120             shell=False,
121         )
122
123
124     def main():
125         """Run the original blood donation demonstration."""
```

Code scanned:

Total lines of code: 118

Total lines skipped (#nosec): 0

Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:

Total issues (by severity):

Undefined: 0

Low: 2

Medium: 0

High: 0

Total issues (by confidence):

Undefined: 0

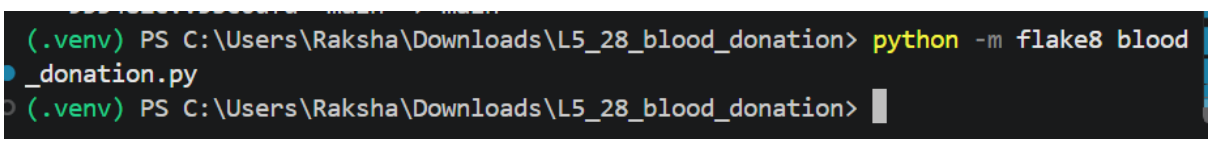
Low: 0

Medium: 0

High: 2

Files skipped (0):

Flake8 output:



Flake8 completed with no findings.

Identified Issues and Fixes:

Issue	Type	Line(s)*	Description	Fix Approach
Shell command injection — B605	Security: High	65	os.system() constructs a shell command using potentially unsafe phone numbers.	Validated phone numbers and replaced it with subprocess.run() using an argument list and shell=False.
Insecure temporary file — B306	Security: Medium	60	tempfile.mktemp() generates a filename without securely creating the file, introducing a race condition.	Used tempfile.mkstemp() and closed the file descriptor with os.close().
Hardcoded password — B105	Security: Low	5	An unused password was stored directly in the source code.	Removed the unused OFFICER_PASSWORD constant.
Assertion used for validation — B101	Robustness: Low	18	Validation using assert is removed when Python runs with optimization enabled.	Replaced it with an explicit check that raises ValueError.
Mutable default argument — W0102	Bug	17	notes=[] is shared across function calls, potentially mixing donor notes.	Changed the default to None and initialized a new list inside the function.
Broad exception handling — W0718	Maintainability	45	except Exception can hide unexpected programming errors.	Replaced it with the specific exception KeyError.
Nonstandard naming — C0103	Readability	17–65	Function and parameter names used camelCase instead of snake_case.	Renamed functions and parameters to snake_case and updated their references.
Missing docstrings — C0114, C0116	Documentation	1 and function definitions	The module and functions lacked descriptions.	Added a module docstring and

Issue	Type	Line(s)*	Description	Fix Approach
				descriptive function docstrings.
Repeated comparisons — R1714	Readability	50	Multiple equality checks tested membership in a fixed collection.	Replaced them with blood_group in {"O-", "AB+"}.
Line too long — E501	Style	7	The stock dictionary exceeded the 79-character limit.	Formatted the dictionary across multiple lines.
Excess blank lines — E303	Style	58	Too many blank lines appeared before a function.	Used two blank lines between top-level functions.
Inline comment spacing — E261	Style	80	There was insufficient spacing before an inline comment.	Added two spaces before the comment.
Missing operator spacing — E225	Style	81	The assignment operator lacked surrounding spaces.	Added spaces around =.

Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest issues were formatting, naming conventions, and missing docstrings because they involved straightforward changes. Replacing the mutable default argument with None was also simple once I understood that a default list is shared across function calls.

The hardest fixes were replacing os.system() and tempfile.mktemp(). These required understanding command injection and temporary-file race conditions while preserving the helpers' intended functionality.

2. Did the static analysis tools report any false positives? If so, describe one example.

No definite false positives were confirmed. However, Bandit reported B404 and B603 after os.system() was replaced with subprocess.run(). These warnings required manual review. The revised code validates phone numbers, passes arguments as a list, and explicitly uses shell=False. Therefore, shell metacharacters are not interpreted as commands. The warnings were retained rather than suppressed. The executable is resolved through PATH, so the execution environment must still be trusted. These findings demonstrate that a warning does not automatically prove that the code contains an exploitable vulnerability.

3. How would you integrate static analysis tools into your actual software development workflow?

I would run Pylint, Flake8, and Bandit during development and before committing changes. Pylint would check code quality, Flake8 would check style, and Bandit would identify potential security issues. I would automate these checks using pre-commit hooks and GitHub Actions on pull requests. High- and medium-severity security findings would receive priority. Any remaining warnings would be reviewed and documented. Functional tests would accompany these checks because static analysis alone cannot establish that the program works correctly.

4. What tangible improvements did you observe after applying the fixes?

The Pylint score improved from 4.81/10 to 10.00/10, and all four original Flake8 findings were resolved. Bandit's original high- and medium-severity findings were eliminated. The final Bandit report retained two low-severity warnings related to subprocess usage. Consistent snake_case names and docstrings improved readability. Replacing the shared default list prevented notes from leaking between calls. Specific exception handling avoided hiding unrelated errors.

Explicit blood-group validation remained active even when Python optimization was enabled.

Secure temporary-file creation reduced race-condition risks. Validated arguments and shell-free subprocess execution reduced command-injection risk. Donation-unit validation also prevented negative or invalid quantities from changing the stock.

The demo produced the same output before and after the changes. This confirms that the demonstrated behavior was preserved, although the SMS and temporary-file helpers were not exercised by the demo.